

Distributed Systems

Lecture2: Design Goals

Mohammad Reza Shakournia

shakournia@ut.ac.ir

School of Electrical and Computer Engineering,
University of Tehran,
Tehran, Iran

Design Goals

- There are four important goals that should be met to make building a distributed system worth the effort.
- A distributed system should:
  - Make resources easily accessible,
  - Hide the fact that resources are distributed across a network,
  - Be open,
  - Be scalable.

+ Dependability
+ Security

Resource Sharing

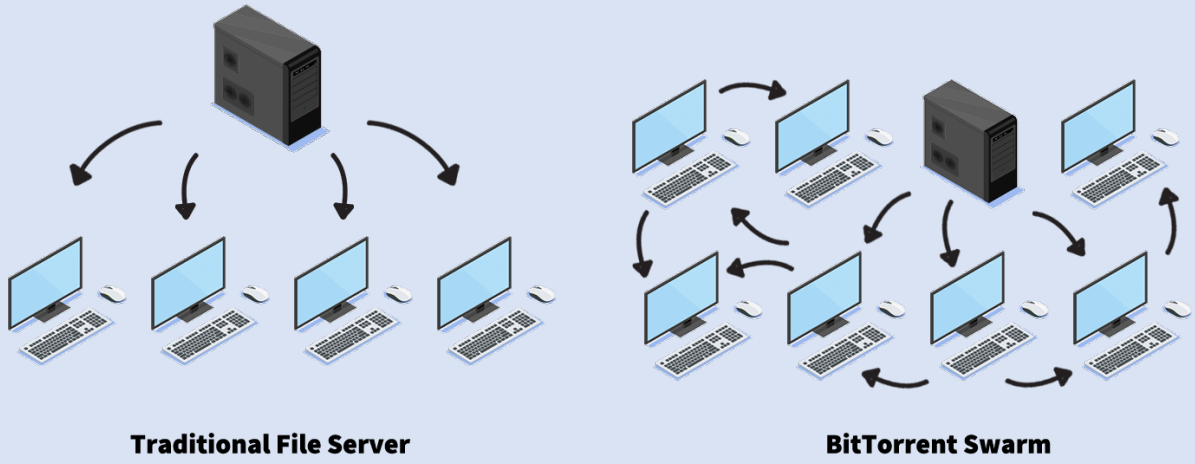
- A key goal of distributed systems is to let users and applications easily access and share remote resources (e.g., devices, storage, data, files, services, networks).
  - For economic reasons (buy printer for each individual).
- By connecting users and resources, distributed systems enable easy collaboration and information exchange.
- Peer-to-peer systems like BitTorrent show successful resource sharing.
- Distributed Systems should enables users and applications to efficiently access and use shared resources across networks, independent of location or ownership.

Note: BitTorrent

Note 1.1: BitTorrent.

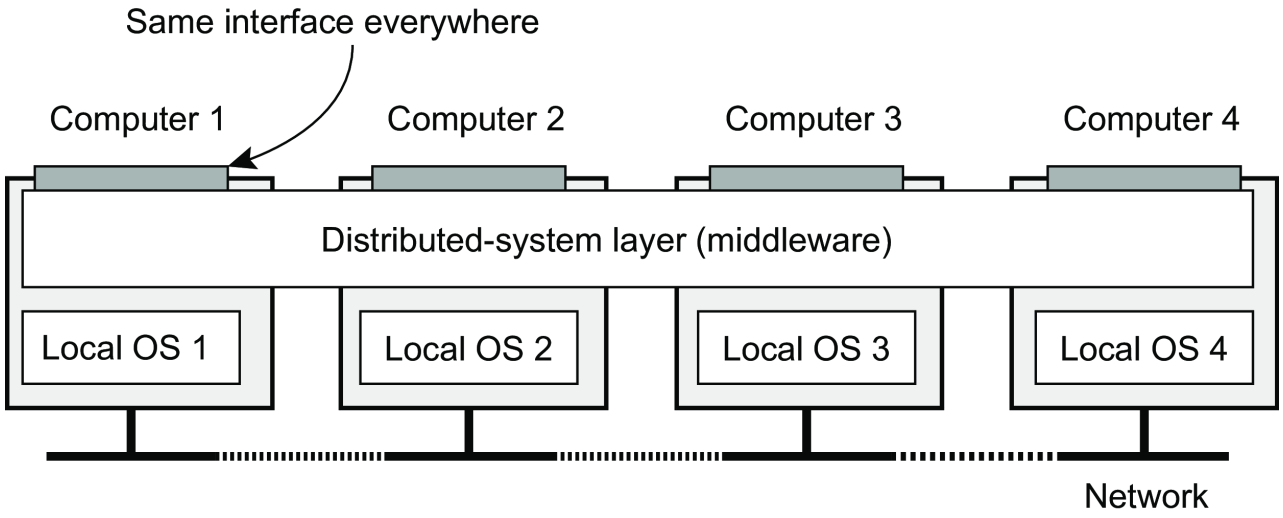
BitTorrent is a decentralized peer-to-peer (P2P) communication protocol used for distributing data and electronic files over the internet.

Instead of downloading a file from a single central server, users download pieces of it from other users (peers) who already have the file, significantly improving speed for large distributions.



Distribution Transparency

- An important goal of a distributed system is to hide the fact that its processes and resources are physically distributed across multiple computers, possibly separated by large distances.
- Try to make the distribution of processes and resources *transparent*, that is, invisible, to end users and applications.
- Achieving distribution transparency is realized through what is known as **middleware**.



Types of Distribution Transparency

- The concept of transparency can be applied to several aspects of a distributed system.

Transparency	Description
Access	Hide differences in data representation and how an object is accessed
Location	Hide where an object is located
Relocation	Hide that an object may be moved to another location while in use
Migration	Hide that an object may move to another location
Replication	Hide that an object is replicated
Concurrency	Hide that an object may be shared by several independent users
Failure	Hide the failure and recovery of an object

Transparency is impossible?!

- Even though transparency is usually desirable, hiding all distribution details can be harmful.(e.g., Different time zone for daily mail.)
- Full transparency is impossible because physical limits and network delays.(e.g., San Francisco–Amsterdam < 35ms!)
- Greater transparency can hurt performance. (e.g., Retrying too long to hide a temporary server failure can slow the system; sometimes failing fast is better.)
- Guarantee that several replicas, located on different continents, must be consistent all the time. (e.g., Propagate all changes to all copies in several continent!)

*Distribution transparency is a good design goal, but it must be balanced with performance and understandability, because achieving full transparency can be **very costly**.*

Openness

- An open distributed system is essentially a system that offers components that can easily be used by, or integrated into other systems.
- Often consist of components that originate from elsewhere.
- **Standard Specification is needed.**
  - Using standards that specify how services are called.(via Interfaces)
  - A well-defined interface lets any process use a service regardless of its implementation.
  - Proper specifications are complete and neutral.
  - Completeness and neutrality are important for interoperability and portability.

Openness

Definition 1.3. Interoperability

Interoperability characterizes the extent by which two implementations of systems or components from different manufacturers can co-exist and work together by merely relying on each other's services as specified by a common standard.

Definition 1.4. Portability

Portability characterizes to what extent an application developed for a distributed system A can be executed, without modification, on a different distributed system B that implements the same interfaces as A.

- An open distributed system should also be extensible.(e.g., plug-ins for Web browsers)

Configurable Policy!

- **Old Monolithic Systems Problem:** changing one part affects the whole system, so the system becomes hard to extend and tends to be closed.
- A flexible open distributed system is built from small, replaceable components with well-defined interfaces.
  - So changing a distributed system become possible.
- The need for changing a distributed system is often caused by a component that does not provide the optimal policy for a specific user or application.(e.g., caching in Web browsers)
  - Where is data to be cached?
  - When the cache fills up, which data is to be removed?
  - Does each browser make use of a private cache?
  - When does a browser check if cached data is still up-to-date?
- What we need is a separation between policy and mechanism. Provide **stable mechanisms**, but **make policies configurable** or even pluggable.

Dependability

- Dependability means how much a system can be trusted to work correctly.
- It's more complex because **partial failures are always happening!**
  - *Some components fail while the overall system still seems fine*
- A key goal is to hide failures and recovery through fault tolerance.
- Dependability in distributed systems includes key requirements such as:
  - Availability,
  - Reliability,
  - Safety,
  - Maintainability.

Note: Fault-Tolerance Metrics

Note 1.2: Fault-Tolerance Metrics.

Maintainability refers to how easily a failed system can be repaired. A highly maintainable system may also show a high degree of availability, especially if failures can be detected and repaired automatically. However, as we shall see, automatically recovering from failures is easier said than done.

Traditionally, fault-tolerance has been related to the following three metrics:

- **Mean Time To Failure (MTTF):** The average time until a component fails.
- **Mean Time To Repair (MTTR):** The average time needed to repair a component.
- **Mean Time Between Failures (MTBF):** Simply MTTF + MTTR.

Faults, Errors, Failures

Definition 1.5. Failure

A system is said to **fail** when it cannot meet its promises. In particular, if a distributed system is designed to provide its users with several services, the system has failed when one or more of those services cannot be (completely) provided.

Definition 1.6. Error

An **error** is a part of a system's state that may lead to a failure. For example, when transmitting packets across a network, it is to be expected that some packets have been damaged when they arrive at the receiver. (e.g., reading a 1 instead of a 0)

Definition 1.7. Fault

The cause of an error is called a fault. Clearly, finding out what caused an error is important. For example, a wrong or bad transmission medium may easily cause packets to be damaged.

Fault Types

- **Transient faults**
  - Occur once and then disappear on their own.
  - Usually fixed by retrying the operation (second attempt often succeeds).
  - Example: a brief external disturbance (e.g., something blocking a wireless/microwave link).
- **Intermittent faults**
  - Appear, vanish, and then reappear unpredictably.
  - Very hard to diagnose because the system may look fine during inspection.
  - Example: a loose connector/contact causing occasional failures.
- **Permanent faults**
  - Persist until the faulty component is repaired/replaced or the bug is fixed.
  - Do not go away with retries; they remain consistently present.
  - Examples: burnt-out hardware, software bugs, disk-head crashes.

*A dependable system must also ensure **confidentiality** (only authorized access) and **integrity** (only authorized changes).*

Security

- A distributed system that is not secure, is not dependable.
- Distributed systems is secure, by ensuring **confidentiality** and **integrity** through controlled access to information and resources.
- both of which are directly coupled to authorized disclosure and access of information and resources.
- **Authorization** checks whether an identified entity has the right permissions, which requires **authentication** to verify that the claimed identity is real.

Even with correct authentication, limited authorization helps reduce potential damage from untrustworthy users

Scalability

- Considering scalability in the design phase means you have planned for your system’s future growth.
- Scalability of a system can be measured along at least three different dimensions:
  - Size scalability,
  - Geographical scalability,
  - Administrative scalability.

Scaling Techniques

- Scalability issues in distributed systems are often performance limits from server/network capacity
- It can be solved by **scaling up** (upgrading hardware) or by **scaling out** (adding more machines) using three main techniques:

  ① Hide Communication Latency,

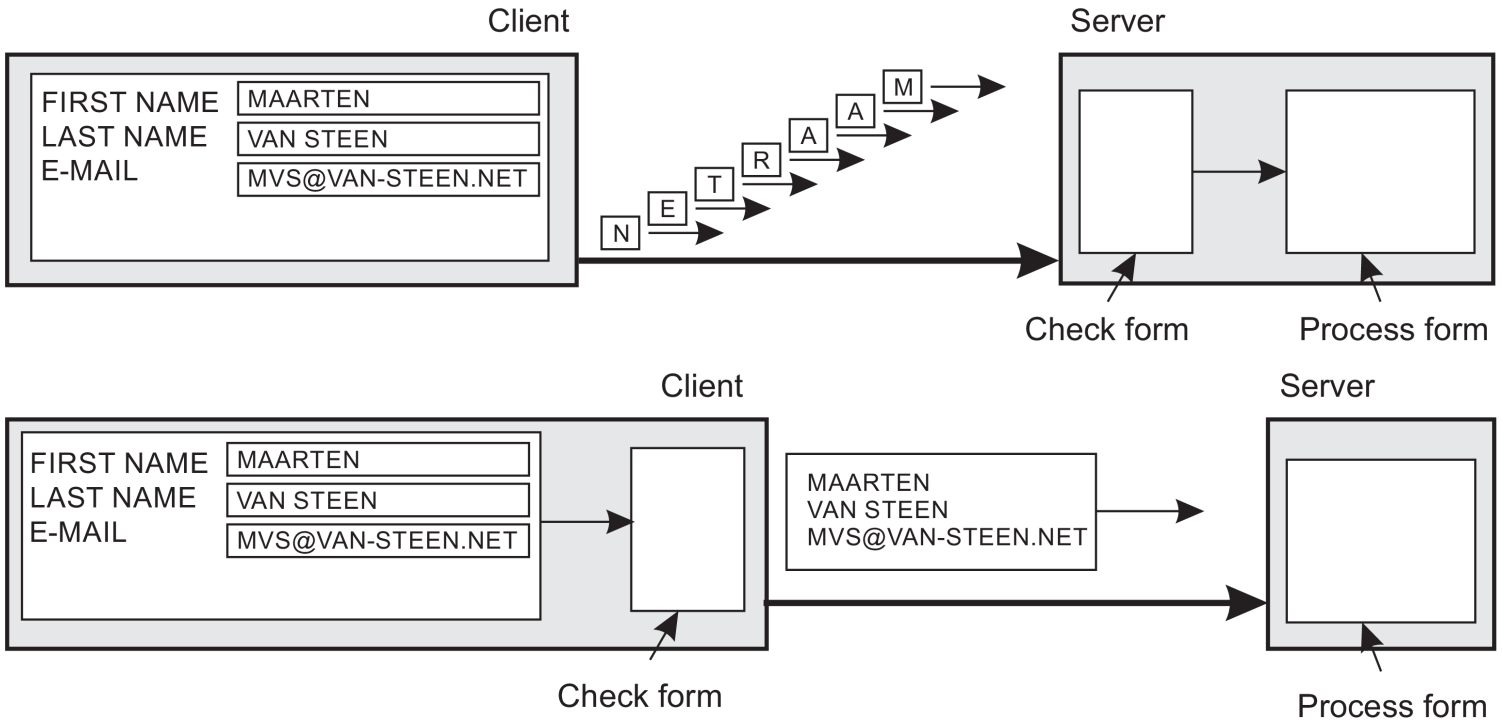
  ② Distribute Work,

  ③ Replicate.

Scaling Tech./ Hiding Communication Latencies

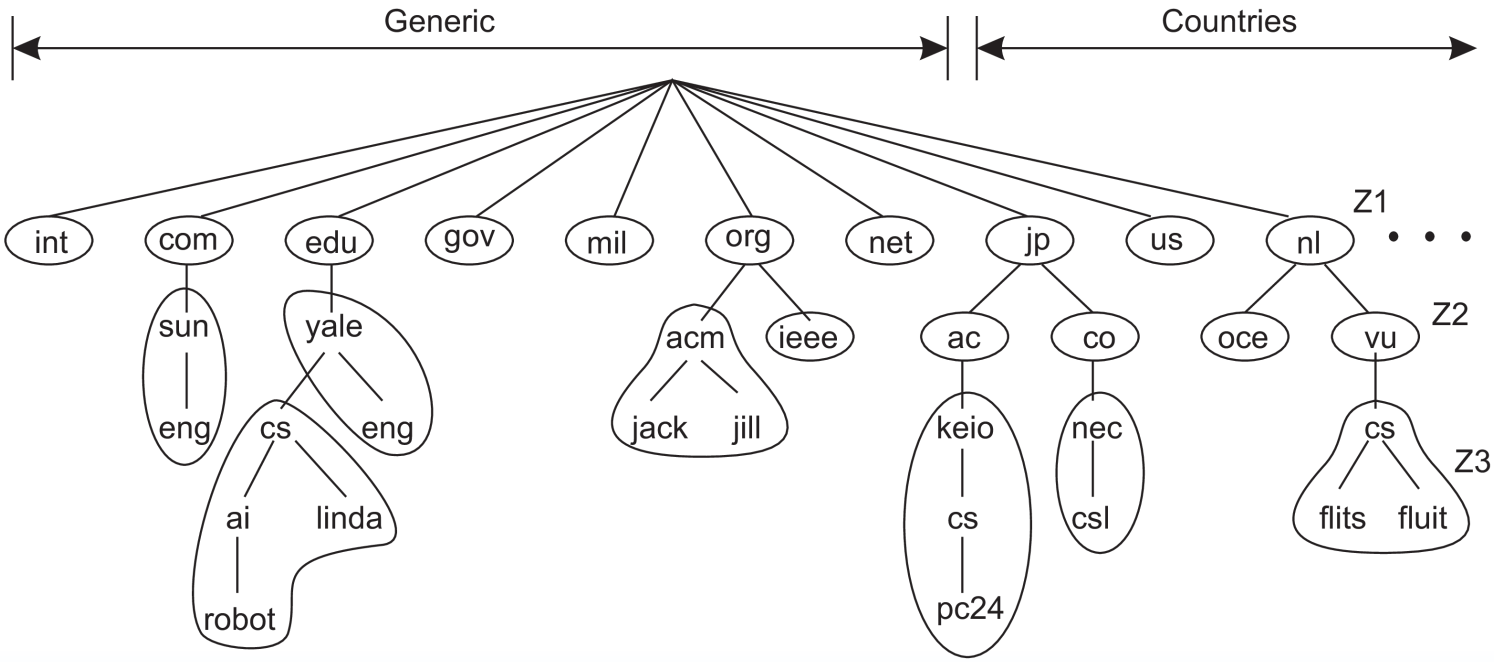
- Hiding communication latency improves geographical scalability by avoiding idle waiting for remote replies.
  - Using **asynchronous communication** (do other work and handle the reply when it arrives),
  - or **Multi-threading** (one thread waits while others continue).
- For applications that cannot make effective use of asynchronous communication:
  - Moving some server work to the client,
  - Ship the code for filling in the form to the client.

Scaling Tech./ Hiding Communication Latencies



Scaling Tech./ Partitioning and Distribution

- Partitioning and distribution scale a system by splitting a resource into smaller parts and spreading them across machines.
- For example, in DNS, where the hierarchical name space is divided into zones served by different name servers that forward queries step-by-step until the host's network address is returned.



Scaling Tech./ Replication

- Scalability problems often appear in the form of performance degradation.
- It is generally a good idea to actually replicate components or resources.
  - Increases availability.
  - Helps to balance the load between components, leading to better performance.
  - In geographically distributed systems, placing a nearby replica reduces many of the communication latency problems.
- **Caching** is a special form of replication.
  - **Drawback: Consistency**

Wrap-Up + Next Lecture

In the next lecture, we will discuss the classification of distributed systems:

- High-performance distributed computing,
- Distributed information systems.